

SENSE-LINK — Technical Documentation

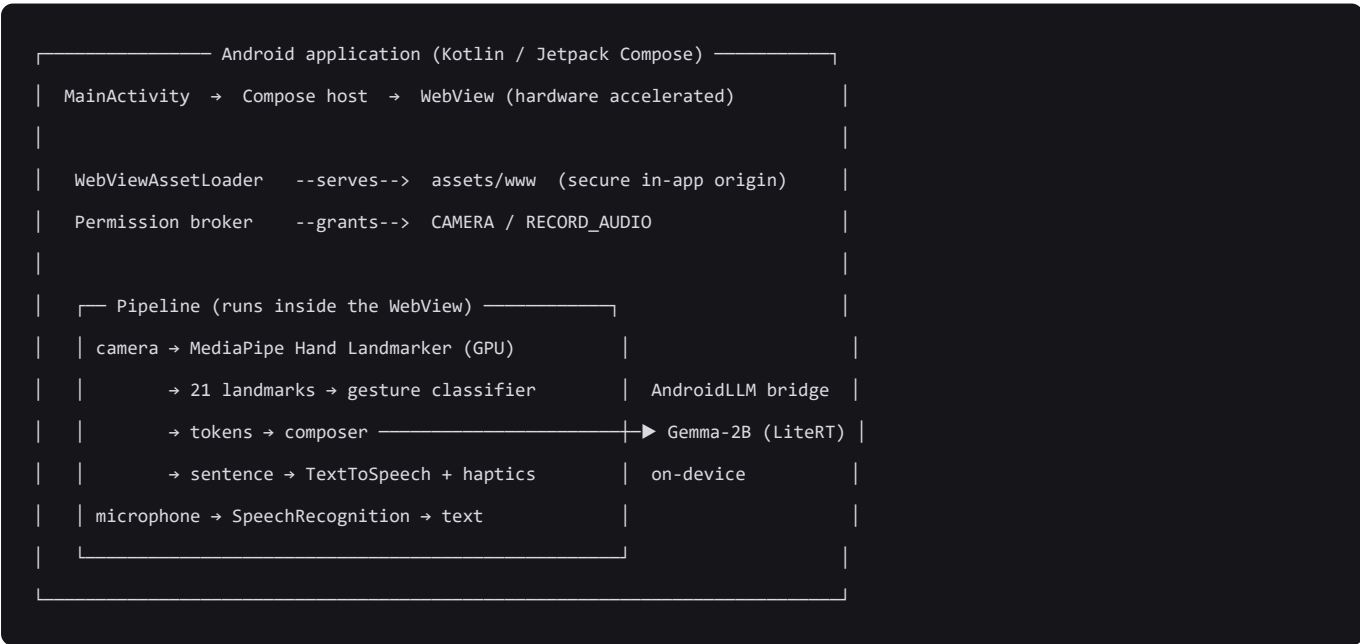
Engineering reference for the SENSE-LINK Android application: a fully on-device, offline sign-to-speech and speech-to-text system. This document covers the architecture, component specifications, the JavaScript-native interface, the on-device language-model runtime, the offline asset model, build and deployment, and the configurable parameters.

1. System overview

SENSE-LINK is a single-activity Android application (Kotlin + Jetpack Compose) that hosts a web-based processing pipeline inside a hardware-accelerated `WebView` and augments it with native capabilities through a JavaScript bridge. All processing — hand tracking, gesture classification, sentence composition, speech synthesis and recognition — executes locally on the device. No network connection is required at runtime; all web assets, model runtimes and fonts are bundled and served from an in-application origin.

Property	Value
Application ID	<code>com.kraftshala.senselink</code>
Version	1.0 (versionCode 1)
minSdk / targetSdk / compileSdk	26 / 34 / 34
Orientation	Portrait (locked)
Network at runtime	None required (optional one-time model download only)

2. Architecture



The application is layered as: (a) a native shell that manages the window, the `WebView`, asset serving and permissions; (b) a web pipeline that performs vision, classification, composition and speech; and (c) a native

model runtime reached over a JavaScript bridge. The only in-device process boundary is that bridge.

3. Technology stack and dependencies

Layer	Technology	Version
Language / UI	Kotlin, Jetpack Compose	1.9.24 / BOM 2024.06.00
Activity integration	<code>androidx.activity:activity-compose</code>	1.9.0
Core	<code>androidx.core:core-ktx</code>	1.13.1
WebView asset serving	<code>androidx.webkit:webkit</code>	1.11.0
On-device LLM	<code>com.google.mediarpipe:tasks-genai</code> (LiteRT)	0.10.14
Hand tracking	MediaPipe Tasks Vision (WebAssembly)	0.10.14
Build	Gradle / Android Gradle Plugin	8.10.2 / 8.5.2
Compose compiler extension	—	1.5.14
Build JDK	JDK	17+

Java/Kotlin compile target is JVM 17.

4. Application shell

4.1 Activity and Compose host

`MainActivity` extends `ComponentActivity`. In `onCreate` it sets `FLAG_KEEP_SCREEN_ON`, requests the required runtime permissions, enables WebView remote debugging, and calls `setContent` with a composable that hosts the WebView through an `AndroidView` interop node. The activity is locked to portrait and declares `configChanges` for orientation, screen size, keyboard and UI mode so it is not recreated on configuration changes (which would tear down the camera session).

4.2 WebView configuration

Setting	Value	Rationale
<code>javascriptEnabled</code>	true	Pipeline is JavaScript.
<code>domStorageEnabled</code>	true	Session state.
<code>mediaPlaybackRequiresUserGesture</code>	false	Camera stream autoplay.
<code>allowFileAccess</code>	false	Assets are served via the loader, not <code>file://</code> .
<code>allowContentAccess</code>	false	No content-provider surface.
Background colour	<code>#0E0E10</code>	Avoids flashes before first paint.

4.3 Local asset serving

Assets are served through `WebViewAssetLoader` mapped to the default reserved origin `https://appassets.androidplatform.net/assets/`. The page is loaded from `https://appassets.androidplatform.net/assets/www/index.html`.

A custom `WebViewAssetLoader.PathHandler` returns each asset with an explicit `Content-Type`. This is required because the default handler mislabels module and WebAssembly files, which breaks ES-module import and WebAssembly instantiation. The handler also sets `Access-Control-Allow-Origin: *`.

Extension	Content-Type
<code>.js</code> , <code>.mjs</code>	<code>text/javascript</code>
<code>.wasm</code>	<code>application/wasm</code>
<code>.html</code>	<code>text/html</code>
<code>.css</code>	<code>text/css</code>
<code>.json</code>	<code>application/json</code>
<code>.woff2</code> / <code>.woff</code> / <code>.ttf</code>	<code>font/woff2</code> / <code>font/woff</code> / <code>font/ttf</code>
<code>.task</code> , <code>.bin</code> , <code>.data</code>	<code>application/octet-stream</code>
other	<code>application/octet-stream</code>

Text types are returned with UTF-8 encoding. Asset requests are intercepted in `WebViewClient.shouldInterceptRequest`.

4.4 Runtime permission broker

Media permissions are handled in two places:

1. **Up front.** `onCreate` requests any missing `CAMERA` / `RECORD_AUDIO` permissions via an `ActivityResultContracts.RequestMultiplePermissions` launcher.

1. **On demand.** When the page calls `getUserMedia`, the WebView raises `WebChromeClient.onPermissionRequest`. The broker maps each requested web resource (`RESOURCE_VIDEO_CAPTURE` → `CAMERA`, `RESOURCE_AUDIO_CAPTURE` → `RECORD_AUDIO`) to its OS permission. If all required OS permissions are held, the request is granted immediately; otherwise the missing permissions are requested and the web request is granted from the result callback once the user approves.

The host activity is resolved by unwrapping the Compose `Context` chain (`ContextWrapper.baseContext`) so the broker is reachable regardless of how the host context is wrapped. If no activity is found, the handler falls back to granting the requested resources directly.

4.5 Lifecycle

`onPause` and `onResume` are forwarded to the WebView so the camera and microphone are released when the app is backgrounded and re-acquired when foregrounded. `onDestroy` closes the model runtime and destroys the WebView.

5. Vision subsystem

Hand tracking uses the MediaPipe Hand Landmarker task running on the GPU delegate. Inference is driven from a `requestAnimationFrame` loop calling `detectForVideo`. Each processed frame yields 21 three-dimensional landmarks, which are drawn as an overlay. Per-frame latency is tracked with an exponential moving average (`latAvg = latAvg*0.85 + ms*0.15`) and displayed live.

Parameter	Value
Task	Hand Landmarker (float16)
Delegate	GPU
Running mode	VIDEO
Max hands	1
Min hand-detection confidence	0.5
Min tracking confidence	0.5

6. Gesture classification

Classification is geometric and stateless per frame. From the 21 landmarks it derives finger extension (fingertip-to-wrist distance compared against the PIP and MCP joints), thumb extension (lateral thumb-tip to index-base distance), and pinch/contact (thumb-tip to index-tip distance, normalised by hand scale). The derived flags map to a fixed lexicon.

Gesture	Token	Gloss
Open palm	HELLO	hello
Fist	HELP	help
Point up	WATER	water
Victory	PLEASE	please
Thumbs up	YES	yes
OK sign	COFFEE	coffee
Pinch	GO	to go

A gesture must be held for a number of consecutive frames before it commits. The hold count scales with the confidence setting: `requiredFrames = round(COMMIT_FRAMES * (0.6 + confidence * 0.8))`. A cooldown prevents the same token repeating on consecutive frames.

Parameter	Default
<code>COMMIT_FRAMES</code>	6
<code>TOKEN_COOLDOWN</code>	1100 ms
Confidence (slider 0.30–0.95)	0.75
Pinch/contact threshold	0.35 × hand scale
Thumb-extension threshold	0.55 × hand scale

Every gesture is also exposed as a tappable control, so the identical token path can be driven by touch when the camera is unavailable.

7. Sentence composition

Committed tokens accumulate in a buffer. After an idle interval (1500 ms) the buffer is composed into a sentence by one of two engines.

7.1 On-device language model (primary)

When a model is loaded, the buffer is sent to Gemma-2B (4-bit) through the `AndroidLLM` bridge (see §10). The bridge runs inference on a background thread with an instruction-style prompt that requests a single natural, polite sentence in the target locale, and returns the result asynchronously. The UI source tag reads `GEMMA-2B`.

7.2 Deterministic composer (fallback)

A rule-based composer maps common token combinations to fixed phrases and otherwise assembles a grammatical sentence from token glosses (lead word, requested items joined with conjunctions, polite suffix, capitalisation and terminal punctuation). It runs synchronously and requires no model. The UI source tag reads `GEMMA-INT4`.

7.3 Selection and watchdog

The LLM path is used when `AndroidLLM.isReady()` returns true; otherwise the deterministic composer runs. A 6000 ms watchdog falls back to the deterministic composer if the model does not return in time, bounding the user-visible latency. The fallback path also applies a short presentation delay (420 ms) before revealing the sentence.

8. Speech I/O

Output uses the Web Speech synthesis API. A voice is selected by matching the configured locale, then an optional voice-profile preference (male / female / neutral) by name pattern. Playback rate is configurable (0.5×–2.0×). Each utterance is paired with start and completion haptics.

Input uses the Web Speech recognition API (`interimResults` on, `continuous` off, language set to the configured locale). Final transcripts are displayed as incoming text with a haptic acknowledgement. (A native `SpeechRecognizer` bridge with an offline language pack is planned for runtimes where the embedded recognizer is unavailable.)

9. Haptics and sensor fusion

Haptic feedback uses `navigator.vibrate` with distinct patterns.

Event	Pattern (ms)
Token captured	18
Sentence start	[40, 55, 40]
Sentence complete	170
Workspace cleared	[30, 40, 30, 40, 30]

Shake-to-clear uses a window-averaging accelerometer detector: the acceleration magnitude is averaged over a sliding window of 10 samples; an average above 22 (rate-limited by a 1400 ms cooldown) clears the workspace.

10. JavaScript-native bridge API

The native object is injected as `window.AndroidLLM`. Methods annotated with `@JavascriptInterface`:

Method	Returns	Description
<code>isReady()</code>	boolean	True when a model is loaded and ready.
<code>status()</code>	string	Human-readable engine state.
<code>modelPath()</code>	string	Absolute path the runtime resolves for the model file.
<code>compose(id, tokensJson, locale)</code>	void	Runs inference for a token array; result delivered asynchronously.
<code>ensureModel(url)</code>	void	Loads the model if present, otherwise downloads it from <code>url</code> .

Results are delivered back to the page through callbacks the page defines:

Callback	Signature	Description
<code>window.__onLLM</code>	<code>(id, sentence)</code>	Composition result for request <code>id</code> ; empty string signals fallback.
<code>window.__onLLMDownload</code>	<code>(percent, message)</code>	Download progress; <code>percent = -1</code> signals an error.

Callbacks are invoked on the UI thread via `WebView.evaluateJavascript`; strings are JSON-quoted.

11. On-device LLM runtime

Model file resolution. The runtime first checks the application's external files directory (`getExternalFilesDir(null)/gemma.task`), then the private storage directory (`filesDir/llm/gemma.task`). A file smaller than 1 MB is treated as absent.

Initialisation. On startup, if a model file is present, the runtime builds `LlmInference.LlmInferenceOptions` (`setModelPath` , `setMaxTokens(512)`) and creates the `LlmInference` engine on a single-thread background executor. Status transitions through `MODEL NOT INSTALLED` → `LOADING GEMMA...` → `GEMMA 2B • ON-DEVICE` , or `LOAD FAILED` on error.

Inference. `compose` joins the token array into a space-separated string, wraps it in the model's instruction template, calls `generateResponse` , and returns the first non-empty line (quotes stripped). Failures return an empty string, which triggers the deterministic fallback on the page.

Provisioning. The ~1.3 GB model is not shipped in the APK. It is provided either by side-loading into the external files directory, or by `ensureModel(url)` , which streams the file to private storage with progress reporting (64 KB buffer; 30 s connect / 60 s read timeouts; written to a `.part` file and atomically renamed on success), then initialises the engine.

12. Offline asset bundling

`android/tools/fetch-assets.mjs` produces the offline bundle under `android/app/src/main/assets/www/` :

- MediaPipe Tasks Vision ES-module bundle and the SIMD and non-SIMD WebAssembly runtimes.
- The hand-landmark model.
- The web fonts (self-hosted `.woff2` plus a generated stylesheet).

The script downloads each artifact and rewrites the web application's external references to local paths, so the bundle is reproducible. A `--html-only` mode re-applies the reference rewrites without re-downloading. At runtime every asset is served from the in-app origin (\$4.3).

13. Control and data flow

1. A camera frame is processed by MediaPipe, producing 21 landmarks.
2. The classifier derives a gesture; after the hold threshold a token is committed and a token haptic fires.
1. Additional gestures extend the token buffer.
2. After the idle interval the buffer is composed (LLM if ready, otherwise deterministic).
3. The sentence is displayed, synthesised to speech, paired with start/complete haptics, and appended to the on-device session history.
1. In the reverse direction, a recognised speech transcript is displayed as incoming text with a haptic cue.
1. A shake clears the workspace.

14. Permissions and security

- Declared permissions: `CAMERA` , `RECORD_AUDIO` , `VIBRATE` , and `INTERNET` (the last only for the optional one-time model download).

- The WebView loads only first-party content from the in-app origin; file-system and content-provider access are disabled and there is no remote content surface.
- The injected bridge exposes a fixed, explicit method set and carries no general command channel.
- Permissions are requested up front and recoverable on demand (§4.4).

15. Performance characteristics

- **Vision.** GPU-delegated hand-landmark inference; measured per-frame latency is shown live and typically sits in the low tens of milliseconds.
- **Classification.** Constant-time arithmetic over 21 points; negligible.
- **Composition.** Deterministic path is effectively instantaneous; on-device LLM latency is device dependent and bounded by the 6000 ms watchdog.
- **Resource profile.** No continuous network transfer; computation is GPU-delegated, keeping the sustained thermal and power envelope low.

16. Build and deployment

```
cd android
node tools/fetch-assets.mjs          # build the offline asset bundle (fresh checkout / after web edits)
./gradlew :app:assembleDebug         # Windows: .\gradlew.bat :app:assembleDebug
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

The output APK is `android/app/build/outputs/apk/debug/app-debug.apk` . A prebuilt debug APK is also provided under `releases/` . Camera and microphone are granted on first launch.

Component	Version
Gradle	8.10.2
Android Gradle Plugin	8.5.2
Kotlin	1.9.24
Compose BOM	2024.06.00
Compose compiler extension	1.5.14
compileSdk / targetSdk / minSdk	34 / 34 / 26
Build JDK	17+

The SDK location is supplied through `local.properties` (`sdk.dir`) or the `ANDROID_HOME` / `ANDROID_SDK_ROOT` environment variables. Release builds reference `proguard-android-optimize.txt` plus `proguard-rules.pro` , which keeps `@JavascriptInterface` members.

17. Configurable parameters

Parameter	Location	Default
Gesture confidence	UI slider	0.75 (0.30–0.95)
<code>COMMIT_FRAMES</code>	gesture loop	6
<code>TOKEN_COOLDOWN</code>	gesture loop	1100 ms
Compose idle delay	composer	1500 ms
LLM watchdog	composer	6000 ms
Shake window / threshold / cooldown	sensor detector	10 samples / 22 / 1400 ms
Locale	UI	<code>en-IN</code> (8 locales)
Voice profile	UI	neutral (male / female / neutral)
Playback rate	UI	1.0× (0.5×–2.0×)
<code>setMaxTokens</code>	LLM runtime	512
<code>LLM_MODEL_URL</code>	web config	empty (set to enable download)

18. Project structure

<code>index.html</code>	Web application (UI + pipeline)
<code>android/</code>	
<code>app/src/main/</code>	
<code>AndroidManifest.xml</code>	
<code>java/com/kraftshala/senselink/</code>	
<code>MainActivity.kt</code>	Compose host, WebView, asset serving, permission broker
<code>LlmBridge.kt</code>	On-device Gemma-2B runtime and bridge
<code>assets/www/</code>	Offline bundle (web app, MediaPipe, model runtime, fonts)
<code>res/</code>	Theme, colours, launcher icon
<code>tools/fetch-assets.mjs</code>	Offline-asset bundler
<code>docs/</code>	
<code>TECHNICAL_DOCUMENTATION.md</code>	This document
<code>THEME.md</code>	Visual design system
<code>releases/</code>	Prebuilt debug APK